



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Cheatsheet Completo: Angular (Arquitectura Moderna v17+, TypeScript Esencial, Reactive Forms, Signals & RxJS)

Resumen Ejecutivo

Esta guía técnica y cheatsheet de preparación para exámenes proporciona una referencia completa y profunda sobre el framework **Angular** en sus versiones modernas (v17+). Incluye los fundamentos avanzados de **TypeScript** aplicados a Angular, arquitectura basada en **Componentes Standalone**, el nuevo sintaxis de **Control Flow**, la API de **Signals**, la gestión avanzada de **Reactive Forms** (con `FormArray` y validadores custom), servicios HTTP e inyección de dependencias, operadores esenciales de **RxJS**, y enrutamiento con guardas funcionales.

1. Requisitos de TypeScript Esenciales para Angular

Angular hace un uso intensivo del sistema de tipos estático y meta-programación mediante TypeScript. A continuación se detallan las características indispensables para el desarrollo y evaluación:

1.1 Decoradores y Meta-datos

Los decoradores son funciones anotadas con @ que agregan metadatos y comportamiento a clases, métodos o propiedades.

```
// Decorador de Clase (Define un Componente Standalone)
@Component({
  selector: 'app-estudiante-detail',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule],
  templateUrl: './estudiante-detail.component.html',
  styleUrls: ['./estudiante-detail.component.css']
})
export class EstudianteDetailComponent {}

// Decorador de Servicio (Inyección Singleton)
@Injectable({
  providedIn: 'root'
})
export class EstudianteService {}
```

1.2 Interfaces vs Types (Modelado de Datos & DTOs)

- **interface:** Se utiliza primordialmente para definir contratos de entidades, modelos de datos de dominio y DTOs recibidos de APIs. Es extensible mediante `extends`.
- **type:** Se prefiere para uniones de literales, alias primitivos, tuplas y tipos derivados complejos.

```
// Modelo de Dominio (Interface)
export interface Estudiante {
  readonly id: number;
  nombre: string;
  correo: string;
```

```

    edad?: number; // Propiedad opcional
    estado: EstadoEstudiante;
  }

  // Unión de Literales mediante Type
  export type EstadoEstudiante =
    'ACTIVO' | 'INACTIVO' | 'SUSPENDIDO';

  // Extensión de Interfaces
  export interface EstudianteBecado extends Estudiante {
    porcentajeBeca: number;
    categoria: string;
  }

```

1.3 Genéricos (<T>) en Servicios e HTTP

Los genéricos aseguran el tipado fuerte al consumir APIs dinámicas como `HttpClient`.

```

@Injectable({ providedIn: 'root' })
export class ApiGenericService {
  private http = inject(HttpClient);

  // Método genérico para consultar endpoints
  obtenerPorId<T>(url: string, id: number): Observable<T> {
    return this.http.get<T>(`${url}/${id}`);
  }
}

```

1.4 Tipos de Utilidad de TypeScript (`Partial`, `Omit`, `Pick`)

Fundamentales para operaciones de creación y actualización en formularios.

```

interface Curso {
  id: number;
  sigla: string;
  nombre: string;
  creditos: number;
}

// Creación: No requerimos el 'id' del backend
type CrearCursoDTO = Omit<Curso, 'id'>;

```

```
// Actualización parcial (PATCH): Claves opcionales
type ActualizarCursoDTO = Partial<Omit<Curso, 'id'>>;

// Vistas resumidas o selectores UI
type CursoResumidoDTO = Pick<Curso, 'sigla' | 'nombre'>;
```

1.5 Operadores de Estrictez (strictNullChecks)

```
// Optional Chaining (?.) y Nullish Coalescing (??)
const nombreMayus =
  estudiante?.nombre?.toUpperCase() ?? 'DESCONOCIDO';

// Non-null Assertion Operator (!) - Inicialización externa
@Input() estudianteId!: number;
```

2. Arquitectura de Componentes Standalone (Angular v17+)

Angular v17 prescinde de los `NgModules` por defecto en favor de los **Standalone Components**, simplificando la modularidad y carga.

2.1 Sintaxis de Control Flow Integrada (@if, @for, @switch)

Reemplaza las directivas estructurales clásicas (`*ngIf`, `*ngFor`, `*ngSwitch`) con mejor rendimiento y soporte nativo del compilador.

```
<!-- @if / @else -->
@if (cargando()) {
  <div class="spinner">Cargando datos...</div>
} @else if (error()) {
  <div class="alert alert-danger">{{ mensajeError() }}</div>
} @else {
  <div class="content">Datos cargados con éxito</div>
}

<!-- @for con rastreo obligatorio (track) -->
<ul class="lista-estudiantes">
  @for (item of listaEstudiantes(); track item.id;
    let i = $index, count = $count) {
```

```

    <li>
      <span>#{{ i + 1 }} de {{ count }}: {{ item.nombre }}</span>
    </li>
  } @empty {
    <li class="empty-msg">No hay estudiantes registrados.</li>
  }
</ul>

<!-- @switch -->
@switch (rolUsuario()) {
  @case ('ADMIN') { <app-panel-admin /> }
  @case ('PROFESOR') { <app-panel-profesor /> }
  @default { <app-panel-estudiante /> }
}

```

■ **Atención Examen:** En `@for`, la cláusula `track` es **estrictamente obligatoria** en Angular moderno. Proporcionar la clave única (ej: `track item.id` o `track $index`) evita re-renderizados completos del DOM.

2.2 Enlace de Datos (Data Binding)

1. **Interpolación:** `{{ expresión }}` (evalúa texto en la plantilla).
2. **Property Binding:** `[property]="variable"` (envía valor del componente al DOM/Input).
3. **Event Binding:** `(event)="método($event)"` (captura eventos del DOM/Output).
4. **Two-Way Binding:** `[(ngModel)]="variable"` (requiere `FormsModule`).

2.3 Comunicación entre Componentes

Angular soporta tanto los decoradores tradicionales como las nuevas APIs funcionales basadas en Signals (`input()`, `output()`, `model()`).

```

import {
  Component, Input, Output, EventEmitter,
  input, output, model
} from '@angular/core';

@Component({
  selector: 'app-tarjeta-estudiante',
  standalone: true,

```

```

    template: `
      <div class="card">
        <h3>{{ estudiante().nombre }}</h3>
        <button (click)="seleccionar()">Seleccionar</button>
      </div>
    `
  })
  export class TarjetaEstudianteComponent {
    // Input requerido de solo lectura (Signal)
    estudiante = input.required<Estudiante>();

    // Output basado en Signal/RxJS EventEmitter
    onSeleccionar = output<number>();

    // Two-way binding moderno con model()
    contador = model<number>(0);

    seleccionar() {
      this.onSeleccionar.emit(this.estudiante().id);
      this.contador.update(val => val + 1);
    }
  }
}

```

```

<!-- Componente Padre consumiendo a TarjetaEstudiante -->
<app-tarjeta-estudiante
  [estudiante]="estudianteActual"
  (onSeleccionar)="procesarSeleccion($event)"
  [(contador)]="totalVistas" />

```

2.4 Ciclo de Vida del Componente (Lifecycle Hooks)

1. **constructor()**: Inyección de dependencias (no realizar peticiones HTTP ni operaciones pesadas).
2. **ngOnChanges(changes: SimpleChanges)**: Se ejecuta cuando cambia una propiedad `@Input()` decorada.
3. **ngOnInit()**: Inicialización del componente, suscripciones y peticiones iniciales.
4. **ngDoCheck()**: Detección de cambios personalizada.
5. **ngAfterViewInit()**: Inicialización de vistas hijas decoradas con `@ViewChild`.

6. **ngOnDestroy()**: Limpieza de suscripciones u timers para evitar **fugas de memoria** (*memory leaks*).

3. Inyección de Dependencias (DI) & Servicios HTTP

3.1 Declaración e Inyección de Servicios

Angular utiliza un sistema jerárquico de Inyección de Dependencias.

```
@Injectable({
  providedIn: 'root' // Singleton en toda la app
})
export class EstudianteService {
  private http = inject(HttpClient); // Inyección inject()
  private apiUrl = 'https://api.ucr.ac.cr/v1/estudiantes';

  obtenerEstudiantes(): Observable<Estudiante[]> {
    return this.http.get<Estudiante[]>(this.apiUrl);
  }

  crearEstudiante(dto: CrearCursoDTO): Observable<Estudiante> {
    return this.http.post<Estudiante>(this.apiUrl, dto);
  }
}
```

3.2 Interceptores HTTP Funcionales (**HttpInterceptorFn**)

Permite transformar solicitudes entrantes y salientes (ej. adjuntar tokens JWT de autenticación o capturar errores globalmente).

```
// auth.interceptor.ts
export const authInterceptor: HttpInterceptorFn =
  (req, next) => {
    const tokenService = inject(TokenService);
    const token = tokenService.obtenerToken();

    const authReq = token ? req.clone({
      setHeaders: {
```

```

    Authorization: `Bearer ${token}`
  }
}) : req;

return next(authReq).pipe(
  catchError((error: HttpResponse) => {
    if (error.status === 401) {
      console.error('Sesión no autorizada');
    }
    return throwError(() => error);
  })
);
};

// app.config.ts (Configuración global en standalone)
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptors([authInterceptor])
    )
  ]
};

```

4. Guía Exhaustiva de Reactive Forms (Deep Dive para Examen)

Los **Reactive Forms** (Formularios Reactivos) son un tema central de evaluación. Ofrecen un flujo de datos síncrono, inmutable y totalmente testeable desde la clase TypeScript.

4.1 Clases Fundamentales

- **FormControl**: Rastrea el valor y el estado de validación de un control individual.
- **FormGroup**: Rastrea el valor y el estado de un grupo de controles.
- **FormBuilder**: Servicio ejecutor para sintaxis concisa de creación de formularios.
- **FormArray**: Rastrea el valor y estado de un arreglo de **FormControl** o **FormGroup** dinámicos.

4.2 Ejemplo Completo: Formulario Complejo con Validadores y FormArray

```
import { Component, inject, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import {
  ReactiveFormsModule, FormBuilder, FormGroup,
  FormArray, Validators, AbstractControl, ValidationErrors
} from '@angular/forms';

@Component({
  selector: 'app-registro-matricula',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule],
  templateUrl: './registro-matricula.component.html'
})
export class RegistroMatriculaComponent implements OnInit {
  private fb = inject(FormBuilder);
  formulario!: FormGroup;

  ngOnInit(): void {
    this.inicializarFormulario();
  }

  private inicializarFormulario(): void {
    this.formulario = this.fb.group({
      carnet: ['', [
        Validators.required,
        Validators.pattern(/^[A-Z]\d{5}$/)
      ]],
      datosPersonales: this.fb.group({
        nombreCompleto: ['', [
          Validators.required,
          Validators.minLength(5)
        ]],
        correo: ['', [
          Validators.required,
          Validators.email
        ]],
      }),
      // Custom Validator a nivel de grupo
      seguridad: this.fb.group({
        password: ['', [
          Validators.required,
          Validators.minLength(8)
        ]],
        confirmPassword: ['', [Validators.required]]
      })
    });
  }
}
```

```

    }, { validators: [this.validarPasswordCoincidente] })),
    // FormArray dinámico para cursos
    cursosMatricular: this.fb.array(
      [],
      [Validators.required, Validators.minLength(1)]
    )
  });
}

```

```

// Getter para acceder al FormArray en el Template
get cursosMatricular(): FormArray {
  return this.formulario
    .get('cursosMatricular') as FormArray;
}

```

```

agregarCurso(): void {
  const cursoGroup = this.fb.group({
    sigla: ['', [
      Validators.required,
      Validators.pattern(/^[A-Z]{2}\d{4}$/)
    ]],
    grupo: [1, [
      Validators.required,
      Validators.min(1),
      Validators.max(50)
    ]]
  });
  this.cursosMatricular.push(cursoGroup);
}

```

```

eliminarCurso(index: number): void {
  this.cursosMatricular.removeAt(index);
}

```

```

// Custom Validator Síncrono
validarPasswordCoincidente(
  control: AbstractControl
): ValidationErrors | null {
  const password = control.get('password')?.value;
  const confirmPassword =
    control.get('confirmPassword')?.value;

  if (password && confirmPassword &&
    password !== confirmPassword) {
    return { noCoincidePassword: true };
  }
}

```

```

    return null;
  }

  guardarMatricula(): void {
    if (this.formulario.invalid) {
      this.formulario.markAllAsTouched();
      return;
    }
    const datosEnvio = this.formulario.value;
    console.log('Enviando payload:', datosEnvio);
  }

  cargarDatosDemo(): void {
    this.formulario.patchValue({
      carnet: 'C12345',
      datosPersonales: {
        nombreCompleto: 'María Pérez'
      }
    });
  }
}

```

4.3 Plantilla HTML para Reactive Form y Manejo de Errores

```

<form [formGroup]="formulario"
  (ngSubmit)="guardarMatricula()">
  <!-- Carnet -->
  <div>
    <label>Carnet UCR:</label>
    <input type="text"
      FormControlName="carnet"
      placeholder="Ej: B12345" />
    @if (formulario.get('carnet')?.invalid &&
      formulario.get('carnet')?.touched) {
      <span class="error">
        @if (formulario.get('carnet')?.errors?.['required']) {
          El carnet es obligatorio.
        }
        @if (formulario.get('carnet')?.errors?.['pattern']) {
          Formato inválido (Ej: B12345).
        }
      </span>
    }
  </div>

```

```

<!-- Sub-grupo datosPersonales -->
<div formGroupName="datosPersonales">
  <label>Correo Institucional:</label>
  <input type="email" formControlName="correo" />
  @if (formulario.get('datosPersonales.correo')?.invalid &&
    formulario.get('datosPersonales.correo')?.touched) {
    <span class="error">Ingrese un correo válido.</span>
  }
</div>

<!-- FormArray Dinámico de Cursos -->
<div formArrayName="cursosMatricular">
  <h3>Cursos a Matricular</h3>
  <button type="button"
    (click)="agregarCurso()">
    + Agregar Curso
  </button>

  @for (cursoGroup of cursosMatricular.controls;
    track $index; let i = $index) {
    <div [formGroupName]="i" class="curso-row">
      <input type="text"
        formControlName="sigla"
        placeholder="Sigla (IF0009)" />
      <input type="number"
        formControlName="grupo"
        placeholder="Grupo" />
      <button type="button"
        (click)="eliminarCurso(i)">
        Eliminar
      </button>
    </div>
  }
</div>

<button type="submit"
  [disabled]="formulario.invalid">
  Procesar Matrícula
</button>
</form>

```

4.4 Diferencias Clave: setValue() vs patchValue()

Característica	setValue()	patchValue()
Estructura	Requiere la estructura exacta y completa de todas las claves del FormGroup.	Permite actualizar un subconjunto parcial de claves.
Comportamiento ante claves faltantes	Lanza un Error de Tiempo de Ejecución (Must supply a value for form control with name...).	Ignora las claves no especificadas silenciosamente y sin lanzar excepción.
Caso de uso de Examen	Reinicialización estricta total del formulario.	Rellenado de datos recibidos de un backend en edición parcial (PATCH).

5. Reactividad: Signals vs RxJS (Estado Moderno en Angular)

Angular v17+ introduce **Signals** para un modelo de reactividad granular y reactivo, complementando el poder asíncrono de **RxJS**.

5.1 Angular Signals (Estado Local y Derivado)

```
import { signal, computed, effect } from '@angular/core';

export class ContadorComponent {
  // 1. Writable Signal (Estado Mutable)
  contador = signal<number>(0);

  // 2. Computed Signal (Estado Derivado de Solo Lectura)
  dobleContador = computed(() => this.contador() * 2);

  constructor() {
```

```

    // 3. Effect (Efecto secundario)
    effect(() => {
        console.log(`Contador: ${this.contador()}`);
    });
}

incrementar() {
    // Modificación basada en el valor previo con update
    this.contador.update(val => val + 1);
}
}

```

5.2 Operadores Fundamentales de RxJS para Exámenes

```

import {
    Observable, Subject, BehaviorSubject, of, throwError
} from 'rxjs';
import {
    map, filter, switchMap, mergeMap, catchError,
    debounceTime, distinctUntilChanged, tap
} from 'rxjs/operators';

// 1. BehaviorSubject: Mantiene el último valor
// emitido y requiere un valor inicial.
const usuarioState$ =
    new BehaviorSubject<string>('Invitado');
usuarioState$.next('Juan'); // Emite 'Juan'

// 2. Operador switchMap: Cancela la suscripción
// anterior al recibir un nuevo evento.
busquedaControl.valueChanges.pipe(
    debounceTime(300), // Espera 300ms de inactividad
    distinctUntilChanged(), // Evita duplicados
    switchMap(termino =>
        this.estudianteService.buscar(termino)
    ), // Cancela peticiones previas pendientes
    catchError(err => {
        console.error('Error en búsqueda:', err);
        return of([]); // Retorna Observable vacío
    })
).subscribe(res => console.log(res));

// 3. Operador mergeMap: Ejecuta múltiples
// suscripciones en paralelo sin cancelar.

```

5.3 Tabla Comparativa de Operadores de Aplanamiento RxJS

Operador	Comportamiento	Caso de Uso Ideal
<code>switchMap</code>	Cancela la petición anterior si llega una nueva emisión.	Campos de búsqueda / Autocomplete HTTP.
<code>mergeMap</code>	Modifica y ejecuta todas las peticiones en paralelo sin cancelar nada.	Guardado masivo independiente o descargas concurrentes.
<code>concatMap</code>	Encola las peticiones internas y las ejecuta en secuencia estricta .	Operaciones secuenciales donde el orden de llegada importa.
<code>exhaustMap</code>	Ignora nuevas emisiones hasta que la petición actual termine.	Botón de submit/pago para evitar doble clic o duplicación.

5.4 Interoperabilidad: `toSignal` y `toObservable`

```
import {
  toSignal, toObservable
} from '@angular/core/rxjs-interop';

@Component({ ... })
export class ListaEstudiantesComponent {
  private estudianteService = inject(EstudianteService);

  // Convierte un Observable en Signal para usar
  // directamente en la plantilla sin AsyncPipe
  estudiantesSignal = toSignal(
    this.estudianteService.obtenerEstudiantes(),
    { initialValue: [] }
  );
}
```

6. Enrutamiento, Navegación y Guardas Funcionales

6.1 Configuración de Rutas Standalone y Lazy Loading

```
// app.routes.ts
import { Routes } from '@angular/router';

export const routes: Routes = [
  { path: '', redirectTo: 'estudiantes',
    pathMatch: 'full' },
  {
    path: 'estudiantes',
    loadComponent: () =>
      import('./estudiantes/estudiantes.component')
        .then(m => m.EstudiantesComponent)
  },
  {
    path: 'estudiantes/:id',
    loadComponent: () =>
      import('./estudiante-detalle/estudiante-detalle.component')
        .then(m => m.EstudianteDetalleComponent),
    canActivate: [authGuard] // Guarda de autenticación
  },
  { path: '**', component: NotFoundComponent } // 404
];
```

6.2 Guardas Funcionales (CanActivateFn)

En Angular moderno las guardas basadas en clases están obsoletas (`deprecated`); se utilizan funciones inyectables.

```
// auth.guard.ts
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

export const authGuard: CanActivateFn = (
  route, state
) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.estaAutenticado()) {
```

```

    return true; // Permite la navegación
  }

  // Redirecciona al login si no está autenticado
  return router.createUrlTree(
    ['/login'],
    { queryParams: { returnUrl: state.url } }
  );
};

```

6.3 Obtención de Parámetros con `withComponentInputBinding()`

Al habilitar `withComponentInputBinding()` en el `provideRouter`, los parámetros de la URL se reciben directamente como `@Input()` o `input()`.

```

@Component({ ... })
export class EstudianteDetalleComponent {
  // Recibe automáticamente el parámetro :id definido en la ruta 'estudiantes/:id'
  id = input.required<string>();
}

```

Documento preparado como material de estudio oficial para el curso IF0009 - Desarrollo de Software IV. Escuela de Informática Empresarial, UCR Recinto Paraíso.